

Aula 14 - Definição do Projeto Final: User Stories e Planejamento

Duração: 2 horas

Objetivos:

- Transformar requisitos em User Stories acionáveis
- Implementar um endpoint baseado em User Stories
- Configurar GitHub Projects para gestão ágil

Parte Teórica (30 min)

1. Componentes de uma User Story Eficaz

markdown

Template:

"Como [perfil], eu quero [ação] para [benefício]"

Exemplo:

"Como cliente, eu quero filtrar produtos por categoria para encontrar itens mais rápido"

Critérios de Aceitação:

- [] Filtro funciona via query param `?categoria=ELETRONICOS`
- [] Retorna status 404 se categoria não existir
- [] Performance < 500ms para 1000 produtos

2. Priorização com MoSCoW

Prioridade	Exemplo no Projeto
Must Have	Cadastro de usuários
Should Have	Histórico de pedidos
Could Have	Recomendações personalizadas
Won't Have	Integração com redes sociais

Atividade Prática (1h30min)

Passo 1: Implementação de User Story

Cenário: "Como cliente, eu quero buscar produtos por nome"

1. Adicione método no Repository

java

```
// src/main/java/com/example/projeto/domain/repository/ProdutoRepository.java
public interface ProdutoRepository extends JpaRepository<Produto, Long> {
    List<Produto> findByNomeContaining(String nome); // Query Method
}
```

2. Crie endpoint no Controller

java

```
// src/main/java/com/example/projeto/interface/rest/ProdutoController.java
@GetMapping("/buscar")
public List<Produto> buscarPorNome(@RequestParam String nome) {
    return repository.findByNomeContaining(nome);
}
```

3. Teste de Integração

java

```
// src/test/java/com/example/projeto/interface/rest/ProdutoControllerIT.java
@Test
void deveBuscarProdutosPorNome() throws Exception {
    repository.save(new Produto("Smartphone Xiaomi"));

    mockMvc.perform(get("/produtos/buscar?nome=Xiaomi"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$[0].nome", containsString("Xiaomi")));
}
```

Passo 2: Configuração do GitHub Projects

1. Crie um board no GitHub

- Columns: Backlog | To Do | In Progress | Done

2. Adicione issues como User Stories

markdown

```
**Title:** US-001 - Busca de produtos por nome
**Labels:** must-have, backend
**Tasks:**
- [ ] Implementar query method
- [ ] Criar endpoint GET /produtos/buscar
- [ ] Escrever testes de integração
```

3. Vincule ao código (git commit)

bash

```
git checkout -b feature/us-001-busca-produtos
git add .
git commit -m "Implementa US-001: Busca de produtos por nome"
git push origin feature/us-001-busca-produtos
```

Passo 3: Definição de Definition of Done

Adicione ao README.md:

markdown

```
## Critérios de Aceitação
✓ Código revisado por pelo menos 1 colega
✓ Cobertura de testes > 80% (JaCoCo)
✓ Passa no SonarQube sem bugs críticos
✓ Documentação Swagger atualizada
```

Checklist de Entrega

- Endpoint de busca implementado
- 1 User Story criada no GitHub Projects
- PR aberto vinculado à issue

Tarefa Pós-Aula

1. Escreva 3 User Stories adicionais para seu projeto
2. Configure um webhook entre GitHub e SonarQube

Exemplo de Webhook:

yaml

```
# .github/workflows/sonar-webhook.yml
name: SonarQube Analysis
on: [pull_request]
jobs:
  sonarqube:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Próximos Passos

- Aula 15: Implementação da Sprint 1 com CI/CD
- Aula 16: Refatoração guiada por métricas

Dica Pro:

"Use `@Transaction` em testes de integração para rollback automático"

Recursos:

- [GitHub Projects Docs](#)
- [Spring Query Methods](#)

Pronto para transformar requisitos em código?  